

12 — Benchmarks

Objetivo de este capítulo: probar que Puglit *mejora* — que la calidad de lo que genera el swarm sube con el tiempo y que esa mejora es **medible por evidencia, no por mecanismo**. No alcanza con decir "tiene 75 agentes y un torneo genético"; hay que mostrar que el torneo le gana a un agente solo, que la app **bootea y no tira 5xx**, y que el cerebro (diary, exemplars, skills) hace que el build N+1 sea mejor que el build N.

Una aclaración de honestidad que atraviesa todo el capítulo: distinguimos entre **métricas instrumentadas** (el código que las graba ya existe y está enchufado al pipeline) y **series históricas** (las curvas before/after que requieren correr el batch de 100 builds, que está *en curso*, no cerrado). Donde una tabla muestra números, son los que el instrumento produce; donde decimos "pendiente de serie", es porque el instrumento existe pero todavía no hay suficientes muestras grabadas para publicar una tendencia. No inventamos números de demo.

12.1 — Benchmark Methodology

El criterio rector de Puglit para benchmarks viene de la crítica que dio origen a `web/lib/swarm-metrics.ts`: "*medí por evidencia, no por mecanismo*". La consecuencia práctica es que **ningún benchmark de Puglit es compile-only**. Un `tsc --noEmit` verde prueba que el código *tipa*, no que *corre*. Puglit insiste en un **runtime gate**.

El runtime gate (no compile-only)

El gate real vive en `web/scripts/build-local.mjs`. Corre el pipeline entero localmente y, en vez de confiar en una afirmación del modelo, **mide**:

1. **SQL load gate** — carga `app.sql` + `seed.sql` contra una DB **fresca** en cada intento (spine → app → seed), y repara el DDL que el modelo erró, hasta 5 vueltas (`loadAppSchemaRepairing`). Si el schema no carga, no hay app.
2. **tsc gate con anti-oscilación** — compila, le devuelve a cada archivo erróneo sus propios errores, y vuelve a compilar. Guarda el *best-seen snapshot* (menos errores) y para si dos rondas no mejoran, restaurando el mejor (`repairTs` , `MAX_ROUNDS=6`).
3. **QA + Queen evidence review** — corre `vitest run --coverage`. La Abeja Reina **mide la cobertura real** y, si está por debajo de la barra (`PUGLIT_COVERAGE_BAR` , default 70%), la rebota con feedback específico ("coverage 41% < 70%, faltan estas funciones...") y el equipo escribe más tests. Loop hasta superar la barra o agotar `PUGLIT_QA_ROUNDS`.
4. **SMOKE gate (RUNTIME GATE)** — la pieza central. Bootea la app con `next dev`, descubre las rutas estáticas caminando el árbol `app/`, le pega un GET a cada una (side-effect-free) y **asietta cero 5xx**. La función `smokeTest()` lo resume: `okGate = fail === 0`. Static scans ≠ working software.

SQL load → tsc green → QA coverage ≥ bar → SMOKE: 0 × 5xx
✗ no app ✗ no tipa 🙌 Queen rebota ✓ "does it actually run?"

La frase que está en el comentario del propio script — "*RUNTIME GATE (crítica: 'nada ejecuta la app')*" — es el lema del benchmarking de Puglit: el único veredicto que cuenta es el de una app que **arrancó y respondió**.

Dónde se graban las métricas

Las métricas instrumentadas se graban en la tabla `puglit_metrics` (`web/sql/genetic.sql: id, name, value, meta JSONB, created_at`) vía `recordMetric(name, value, meta)`. La agregación es una media móvil por ventana (`metricRate(name, sinceDays=30) → AVG(value)`), y el tablero headline es `scorecard()`, que consulta cuatro nombres. Todas las escrituras son **fire-and-forget** (`void recordMetric(...).catch(() => {})`): instrumentar el benchmark nunca puede tirar un build.

12.2 — Reference Projects

Puglit se mide contra dos conjuntos de proyectos de referencia: un **litmus test** profundo y un **batch de 100** ideas amplio.

Stayforge — el litmus test

Stayforge (un clon de Airbnb) es la app deliberadamente difícil que ejercita las partes del sistema que un CRUD trivial no toca. Sus números de referencia:

Dimensión	Valor
Archivos generados	80
Tablas	8
Anti-double-booking	Postgres <code>EXCLUDE USING gist</code> (constraint a nivel DB, no chequeo en app)
Pricing	enteros en centavos, determinístico (sin floats)
Refund	por <i>snapshot</i> de la política vigente al momento de reservar
Reviews	doblo-ciego
Bugs reales que expuso el runtime gate	7 (invisibles a un compile verde)

El valor de Stayforge como benchmark no es que se genere — es que el `EXCLUDE` constraint, el pricing en centavos y el refund-by-snapshot son exactamente el tipo de cosa que **compila pero falla en runtime** si está mal. Los **7 bugs** que el gate cazó (sección 12.5) son la prueba empírica de la tesis "compile-only no alcanza".

El batch de 100 — `brain-training-ideas.tsv`

El segundo conjunto es el **100-build brain-training batch** (`infra/train-brain.sh` + `infra/brain-training-ideas.tsv`). Verificado: el TSV tiene exactamente **100 ideas**. La composición real:

Eje	Distribución (verificada en el TSV)
Modo <code>train</code> (ligero: sin repair del adverse, 1 ronda QA)	66
Modo <code>full</code> (torneo completo, 2 rondas QA)	34
Monetización <code>free</code>	81
Monetización <code>suscripcion</code>	11
Monetización <code>pago</code>	8

La diversidad es deliberada y cubre los caminos del código: juegos retro/Atari con **input simultáneo** (Pong W/S + flechas, Asteroids rotar+empujar+disparar a la vez), calculadoras, trackers, verticales de negocio con **agenda sin solapamiento** (Peluquería, Veterinaria, Reservas de Mesa — el mismo patrón anti-double-booking de Stayforge), **dinero** con ledger append-only (Puntos de Lealtad), catálogos, quizzes, y apps que **aprenden** (Akinator, que agrega objeto + pregunta distintiva cuando falla).

`train-brain.sh` es **resumable** (saltea las ideas ya hechas vía `$DONE`), corre el torneo (`/api/genetic/tournament`) y después el build del ganador — *que es donde el cerebro aprende*: graba exemplars, corre QA/adverse y emite métricas. Cada `EVOLVE_EVERY=20` builds dispara una pasada de **SkillOpt** (`/api/admin/evolve-skills`), y al final snapshota el cerebro a git.

Estado honesto: el batch está *en curso*. Las celdas de "rate" de las tablas que siguen muestran *qué se mide y cómo*, no una corrida cerrada de 100/100. Cuando el batch termine, `scorecard()` devuelve las tasas reales sobre $N \approx 100$ muestras.

12.3 — Success Rate — métrica `build_success`

Instrumentada. Se graba en `web/lib/app-builder.ts` al final de cada build:

```
const highIssues = checks.issues.filter((i) => i.severity === "high").length
void recordMetric("build_success", highIssues === 0 ? 1 : 0,
  { files: files.length, issues: checks.issues.length, high: highIssues, repaired })
```

Aspecto	Definición real
Qué cuenta como éxito	El static gate (<code>runSwarmChecks</code>) termina con cero issues de severidad high tras el self-healing
Valor grabado	<code>1</code> (sin highs) / <code>0</code> (con al menos un high)
meta	nº de archivos, total de issues, highs, tablas reparadas
Agregación	<code>metricRate("build_success")</code> → <code>AVG(value)</code> sobre 30 días → tasa 0.0–1.0
Dónde aparece	primera entrada de <code>scorecard()</code>

Nótese que `build_success` se evalúa **después** del self-healing (security repair con frontier escalation + reparación de phantom-tables). Es decir: mide la calidad del *entregable final*, no del primer borrador crudo del modelo chico. La diferencia entre el borrador y el entregable es justamente lo que aporta el swarm.

Honestidad: la tasa `build_success` se llena a medida que corre el batch. El instrumento existe y está enchufado; la serie publicable sale del batch de 100.

12.4 — Build Time

El tiempo de build es **observable** desde dos lugares, ambos verificados en el código:

- `build-local.mjs` loguea el wall-clock de la fase de generación (`const t0 = Date.now()` ... `((Date.now() - t0) / 60000).toFixed(1)` minutos por vuelta de `/advance`).
- `train-brain.sh` loguea el total del batch (`MIN=$(( ( $(date +%s) - START ) / 60 ))`) → "entrenamiento completo: \$ok/\$n builds OK en \${MIN} min", y por idea espera el torneo (hasta ~25 min) y el build del ganador (~30 min) con polling cada 5 s.

Fase	Presupuesto de tiempo (del código)	Cómo se mide
Torneo (3 teams divergen + jurado)	~25 min (300 × 5 s polling)	<code>train-brain.sh</code>
Build del ganador (donde aprende el cerebro)	~30 min (400 × 5 s polling)	<code>train-brain.sh</code>
Total del batch	<code>\${MIN}</code> min, logueado al final	<code>train-brain.sh</code>

Atención al contexto: estos presupuestos son para Ollama **local** con modelos chicos (default `qwen2.5-coder:7b` en `build-local.mjs`) en una caja modesta. No son una promesa de SLA; son los timeouts que el harness tolera. El componente `run_score` (§12.8) es el que mide latencia/costo *por build* y lo empuja hacia abajo con el tiempo.

Honestidad: los tiempos por build se *loguean* pero no se persisten hoy como métrica agregada en `puglit_metrics` (no hay un `recordMetric("build_time_ms", ...)`). El dato vive en los logs del batch; la serie agregada de build-time es **pendiente de instrumentar**.

12.5 — Bug Rate — los 7 bugs reales que el gate encontró

El número más honesto del whitepaper. Sobre **Stayforge**, el runtime gate expuso **7 bugs reales** que un compile verde no veía. Esa es la afirmación central de la metodología "runtime, no compile-only": un sistema que solo chequea tipos los hubiera dejado pasar a los 7.

La taxonomía de bugs que el gate caza (derivada de las reparaciones que el código realmente implementa) deja claro *por qué* compile-only no alcanza:

Categoría de bug	Quién lo caza	Por qué <code>tsc</code> lo deja pasar
Phantom-table (route que hace <code>FROM</code> <code>tabla_inexistente</code>)	<code>swarm-checks</code> + <code>swarm-repair</code> + penalty en <code>objectiveScore</code>	la query es un string; el tipo es <code>string</code> , compila
SQL DDL inválido (ENUM inline, orden de FKs, comas finales)	SQL load gate (<code>loadAppSchemaRepairing</code>)	el <code>.sql</code> no lo ve TypeScript
Spine import faltante (<code>pool</code> / <code>getAuthUser</code> / <code>NextResponse</code> sin import)	<code>ensureSpineImports</code> (fix determinístico)	a veces <code>tsc</code> lo cacha, a veces es <code>ReferenceError</code> 500 en runtime
JSX sobre-escapado (<code>className=\\ "...\\"</code> del JSON del LLM)	<code>unescapeJsx</code>	el archivo "tipa" pero cada página tira 500
"use client" faltante con hooks	<code>ensureUseClient</code>	compila; explota al renderizar
5xx en runtime (cualquier crash al bootear/servir una página)	SMOKE gate	imposible de ver sin ejecutar la app
Secrets / SQLi (patrones inseguros)	<code>swarm-checks</code> (scan de seguridad)	sintácticamente válido

El punto: cada fila es una clase de error que **compila y falla en runtime**. Stayforge materializó 7 instancias concretas de estas clases, y el gate las cazó todas antes de entregar. La **bug rate** que Puglit reporta es, por construcción, la tasa de issues `high` *residuales* tras el gate — es decir, `1 - build_success` (§12.3) sobre la severidad alta.

12.6 — Repair Rate

Puglit no solo detecta: **repara**, y la reparación es bounded (presupuestada) para no entrar en loops ni gastar de más. Las reparaciones instrumentadas:

Reparador	Archivo	Bounded por
Phantom-table (infiere schema del intent)	<code>web/lib/swarm-repair.ts</code>	escalación a frontier con presupuesto por build
Security repair (frontier escalation)	<code>app-builder.ts</code> (<code>repairSecurityWithFrontier</code>)	presupuesto de frontier por build (<code>resetFrontierBudget</code>)
Adversarial bounded-repair de CRITICALs confirmados	<code>web/lib/adversarial-review.ts</code>	solo repara lo cross-validado por ≥ 2 lentes
SQL DDL repair	<code>build-local.mjs</code> (<code>loadAppSchemaRepairing</code>)	5 vueltas máx
tsc repair	<code>build-local.mjs</code> (<code>repairTs</code>)	<code>MAX_ROUNDS=6</code> , para a las 2 rondas sin mejora

La señal de cuánto reparó cada build viaja en el `meta` de `build_success` (`{ ..., repaired }` = nº de tablas reparadas) y en `adverse_verdict` (`{ confirmed, solo, repaired }`). El **repair rate** es, entonces, derivable de `puglit_metrics` : la fracción de builds con `meta.repaired > 0` , y dentro del adverse, la fracción de findings confirmados que terminaron en `repaired` .

Honestidad: la cuenta agregada de "repair rate" no tiene hoy un nombre de métrica dedicado; se obtiene *post-hoc* leyendo el `meta` JSONB de `build_success` y `adverse_verdict` . El instrumento (el campo `repaired` en el `meta`) ya existe; la serie sale del batch.

12.7 — Judge Accuracy — métrica `judge_agreement`

Instrumentada. El torneo genético (`web/lib/tournament.ts` , `judgeBlueprints`) corre un **panel multi-juez** sobre los blueprints de los 3 teams. El acuerdo inter-juez se graba:

```
void recordMetric("judge_agreement",
  verdicts.length > 1 ? topAgree / verdicts.length : 1, { jurors: verdicts.length })
```

Aspecto	Definición real
Qué mide	fracción de jurados que coinciden con el ganador top (<code>topAgree / verdicts.length</code>)
Caso de un solo jurado	1 (acuerdo trivial)
<code>meta</code>	nº de jurados
Métrica gemela	<code>judge_failed</code> (graba 1 cuando un jurado falla — el circuit breaker jury-down → draft)
Veredicto final	blend 60% LLM + 40% <code>objectiveScore</code> (ver §12.2 / <code>swarm-fitness</code>)

`judge_agreement` es el indicador de *salud del jurado*: si el acuerdo es alto, el panel es consistente y el veredicto es confiable; si es bajo, la decisión es ruidosa y el circuit breaker puede caer a draft. No mide si el juez *acertó* contra un ground-truth externo (eso requeriría labels humanos) — mide **consistencia inter-juez**, que es el proxy honesto y barato del que dispone el sistema.

12.8 — Module Reliability

Dos métricas instrumentadas cubren la confiabilidad de lo entregado más allá del static gate.

`adverse_verdict` — el veredicto adversarial

Instrumentada (`app-builder.ts` , vía `web/lib/adversarial-review.ts`). Tres lentes ortogonales (Auditor=lógica, Adversary=abuso/seguridad/reject-bias, Pragmatist=diseño) revisan el entregable, se cross-examinan (≥ 2 lentes = confirmado) y emiten un veredicto que se mapea a número:

Veredicto	Valor grabado
SHIP	1
SHIP-WITH-CAVEATS	0.5
BLOCK	0

`meta = { confirmed, solo, repaired }`. La tasa `adverse_verdict` agregada es un índice de confiabilidad del módulo entregado: cuán seguido el panel adversarial lo deja pasar limpio.

`run_score` — agent-house

Instrumentada. Cada build perfila su traza de llamadas LLM (`web/lib/swarm-profile.ts` + `run-trace.ts` , idea addyosmani/agent-house) y graba:

```
void recordMetric("run_score", profile.score,
  { ...profile.categories, savingsMs: profile.savingsMs, calls: profile.run.calls })
```

Puntúa **Cost / Latency / Reliability / Context** de la corrida y emite fixes rankeados. Es la métrica que hace al swarm *más barato y más rápido* con el tiempo: `run_score` subiendo build a build es la evidencia de que la maquinaria se afina, no solo el producto.

12.9 — Evolution Impact — SkillOpt `val_score` before/after

Acá está el corazón de la tesis "Puglit mejora". **SkillOpt** (`web/lib/skill-evolution.ts` , estilo microsoft/SkillOpt) trata el **skill doc** de cada área como estado *entrenable* (sin tocar pesos) y solo acepta una edición si **gana en un set de validación held-out**.

El mecanismo de medición before/after es explícito:

```
const before = active[area]?.score ?? (await validate(current))
// ... el optimizer propone edits bounded ...
const after = await validate(candidate)
const MARGIN = Number(process.env.PUGLIT_SKILL_MARGIN || 1.5)
if (after > before + MARGIN) { /* ACEPTAR: nueva versión active con val_score = after */ }
else { /* push al rejected-edit buffer */ }
```

Parámetro	Valor real
Validación	<code>validate()</code> promedia <code>objectiveScore</code> sobre 5 tareas frozen (<code>VAL_TASKS</code>)
Set held-out	rentals marketplace, used-goods swipe, live football scores, team task manager, subscription box
Edits bounded	≤ 3 add/delete/replace, net ≤ 500 chars, doc final ≤ 1900 chars (el "learning rate" textual)
Criterio de aceptación	<code>after > before + 1.5</code> — estrictamente mejor por margen (anti noise-chasing)
Anti-regresión	rejected-edit buffer (<code>puglit_skill_rejects</code>) — no se repite una edición rechazada
Artefacto desplegado	el skill doc evolucionado, usado en generación a costo de inferencia cero
Áreas	<code>data, dev, design, business, test</code>

El **before/after** es, por diseño, la prueba de mejora más limpia que tiene Puglit: una edición de skill **no entra al sistema** salvo que suba el `val_score` held-out por más de 1.5 puntos. La tabla `puglit_skills` guarda `(area, version, doc, val_score, status)` con versionado: cada `active` archiva al anterior. La trayectoria de `val_score` por área a lo largo de las versiones **es** la curva de mejora del cerebro.

Honestidad: la curva `val_score` por versión existe en cuanto SkillOpt acepta ediciones (lo dispara `train-brain.sh` cada 20 builds + una pasada final). El instrumento está completo y el gate de aceptación es real; la *serie* de versiones se va llenando con el batch. Si SkillOpt nunca acepta una edición, el sistema cae limpiamente a los seed playbooks (`PLAYBOOK.*`) — un default seguro, no un fallo.

12.10 — Historical Trends — qué se trackea en `puglit_metrics`

El destino de todo lo anterior es una **serie temporal** en `puglit_metrics` . Resumen honesto del estado de instrumentación:

Métrica	Nombre	¿Instrumentada?	¿En <code>scorecard()</code> ?	Estado de serie
Success rate	<code>build_success</code>	✓ <code>app-builder.ts</code>	✓	pendiente de batch
Adverse verdict	<code>adverse_verdict</code>	✓ <code>app-builder.ts</code>	—	pendiente de batch
Run profile	<code>run_score</code>	✓ <code>app-builder.ts</code>	—	pendiente de batch
Judge agreement	<code>judge_agreement</code>	✓ <code>tournament.ts</code>	✓	pendiente de batch
Judge failures	<code>judge_failed</code>	✓ <code>tournament.ts</code>	—	pendiente de batch
SkillOpt val	<code>puglit_skills.val_score</code>	✓ <code>skill-evolution.ts</code>	— (tabla aparte)	crece al aceptar edits
Smoke pass	<code>smoke_pass</code>	⚠ listado pero NO grabado	✓ (esperado)	gap de instrumentación
Ablation	<code>ablation_swarm_win</code>	⚠ listado pero NO grabado	✓ (esperado)	gap de instrumentación

Dos gaps honestos y verificados. `scorecard()` (`swarm-metrics.ts`) lista cuatro nombres — `build_success`, `smoke_pass`, `judge_agreement`, `ablation_swarm_win` — pero un grep sobre todo el codebase confirma que **`smoke_pass` y `ablation_swarm_win` no tienen ningún `recordMetric()` que los escriba:**

- **`smoke_pass`** : el resultado del SMOKE gate **se computa** (`okGate = fail === 0` en `build-local.mjs`) pero hoy se persiste a un *status file* (`writeStatus({ smoke: okGate, ... })`), **no** a `puglit_metrics`. El dato existe; falta el puente `recordMetric("smoke_pass", okGate ? 1 : 0)`. Hasta entonces `metricRate("smoke_pass")` devuelve `{ rate: 0, n: 0 }` — la celda del `scorecard` sale vacía, no falsamente verde.
- **`ablation_swarm_win`** : la ablación (torneo de 3 teams vs un solo agente) es la prueba que justifica la complejidad — *"si el torneo no le gana al agente solo, sobra complejidad"* — pero su `recordMetric` todavía no está cableado en el path de generación.

Reportar esto es exactamente lo que la metodología de Puglit exige: medir por evidencia significa también **declarar dónde todavía no hay evidencia**. Las cuatro métricas del `scorecard` tienen el *lugar* reservado; dos están grabando hoy y dos esperan el puente. Cuando el batch de 100 cierre y los dos puentes (`smoke_pass`, `ablation_swarm_win`) estén cableados, `scorecard()` devuelve el tablero completo sobre $N \approx 100$, y la trayectoria de `val_score` por versión documenta —con números reales— que **Puglit mejora**.

Resumen del capítulo

Pregunta del benchmark	Cómo la responde Puglit	Estado
¿La app corre ?	SMOKE gate: 0 × 5xx tras bootear	instrumentado (status file)
¿El entregable es limpio?	<code>build_success</code> = 0 highs residuales	instrumentado en <code>puglit_metrics</code>
¿El jurado es confiable?	<code>judge_agreement</code> inter-juez	instrumentado en <code>puglit_metrics</code>
¿El panel adversarial lo deja pasar?	<code>adverse_verdict</code> SHIP/CAVEATS/BLOCK	instrumentado en <code>puglit_metrics</code>
¿El swarm se abarata?	<code>run_score</code> (cost/latency/reliability/context)	instrumentado en <code>puglit_metrics</code>
¿El cerebro aprende ?	SkillOpt <code>val_score</code> before/after, gate +1.5 held-out	instrumentado en <code>puglit_skills</code>
¿El torneo justifica su complejidad?	<code>ablation_swarm_win</code>	pendiente de cablear

El runtime gate ya probó su valor empírico: **7 bugs reales en Stayforge** que un compile verde no veía. El resto del aparato de medición está instrumentado y esperando que el batch de 100 lo llene de muestras. Esa es la postura honesta de este whitepaper: el instrumento existe, la tesis es falsable, y las series se publican cuando el batch cierre — no antes.